

Spec — Telegram Architecture: Swappable Environments + Two-Bot Model

Created: 2026-06-20 — Claude.ai Replaces: spec_telegram_architecture_2026-06-20.md
Status: Ready for `_working/` Priority: High — blocks full EC2 validation

Context

mini-moi now runs on two instances: AWS EC2 and Mac. The original Telegram spec assumed EC2 is permanently primary. The actual target is a swappable model: AWS is the default production environment, but the Mac (and eventually a Mac Mini) can be promoted to production when needed — with roughly 12 hours switchover time including testing.

This spec replaces the previous version with a cleaner two-bot model and a role system that supports graceful environment promotion.

Design Decisions (locked)

Swappable production. Either environment can be production. `MINIMOI_ROLE=production` is set on whichever instance is currently active. The other runs as `standby`. Only one instance is production at any time.

Two bots instead of three. Simplified from the original three-bot design:

Bot	Purpose	Direction
<code>minimoi_agent_bot</code>	Conversational — CoS + OpenClaw natural language	Bidirectional
<code>minimoi_system_bot</code>	System content — Curator briefings, German drills, alerts, <code>!ops</code> , feedback, notifications	Mixed

Clean separation: you talk to the agent bot, the system talks to you via the system bot. Commands like `!ops` go to the system bot since they trigger system actions, not conversations.

Production/standby naming. Clearer than primary/secondary — reflects that one instance is active production, the other is available for development, testing, or emergency promotion.

Test bot sets. One test version of each bot for use on the standby instance. Same developer receives both — bot name distinguishes which instance sent a message.

Part 1 — Bot Lane Assignments (Cleanup)

Two-bot model

minimoi_agent_bot — Conversational, bidirectional:

- Chief of Staff natural language interaction
- OpenClaw memory queries and updates
- Design session capture ("note this conversation")
- Anything requiring a back-and-forth exchange

minimoi_system_bot — System content and commands:

- Curator daily briefing delivery
- German session analysis and drills
- Deploy notifications (success/failure)
- `!ops` and any `!` prefixed commands
- Alerts and system health notifications
- Outbound status messages of any kind

Migration from three bots: The previous `minimoi_cmd_bot`, `rvsopenbot`, and `minimoi_agent_bot` collapse into two. Audit all `send_message()` calls and reassign:

- Outbound content → `minimoi_system_bot` token
 - Conversational → `minimoi_agent_bot` token
 - `!ops` → `minimoi_system_bot` (inbound command handler)
-

Part 2 — Test Bot Setup

Robert creates these in BotFather (5 minutes):

Test bot	Mirrors	Purpose
<code>minimoi_agent_bot_test</code>	<code>minimoi_agent_bot</code>	Test conversational flows
<code>minimoi_system_bot_test</code>	<code>minimoi_system_bot</code>	Test briefings, alerts, commands

Same chat ID as production. Bot name in header distinguishes which instance sent the message.

Part 3 — Production/Standby Role System

Environment variable

```
bash

# AWS EC2 .env (production - default)
MINIMOI_ROLE=production

# Mac .env (standby - default)
MINIMOI_ROLE=standby
```

Default is `production` so EC2 needs no special handling beyond what's already in `.env`. Mac explicitly set to `standby`.

Role helper

```
python

import os

def is_production() -> bool:
    """
    Returns True if this instance is the active production instance.
    Default is production - standby must be explicitly set.
    """
    return os.environ.get('MINIMOI_ROLE', 'production') == 'production'

def get_telegram_path() -> str:
    """
    Returns SSM path prefix for Telegram credentials based on role.
    Production uses real bots. Standby uses test bots.
    """
    return 'production' if is_production() else 'test'
```

Startup logging (required)

Every app logs its role on startup. Makes debugging trivial.

```
python
```

```
role = os.environ.get('MINIMOI_ROLE', 'production')
bot_path = get_telegram_path()
app.logger.info(
    f"Starting as {role} instance using {bot_path} Telegram bots"
)
```

Guards on all outbound actions

```
python
```

```
# Curator briefing
if is_production():
    send_telegram_briefing(briefing)
else:
    app.logger.info("Standby – briefing suppressed")

# Scheduled jobs
if is_production():
    run_curator_scoring_loop()
else:
    app.logger.info("Standby – scoring loop suppressed")

# Bot polling startup
role = get_telegram_path()
app.logger.info(f"Bot starting with {role} tokens")
# Bot continues with role-appropriate tokens – never suppressed entirely
# Standby still runs bots so testing is fully functional
```

Standby suppresses: scheduled outbound jobs (briefings, alerts, scoring loops, CoS periodic tasks)

Standby still runs: web app, Telegram bots (with test tokens), manual commands — full testing capability

Part 4 — SSM Parameter Structure

Two paths, matching two roles:

Production (/minimoi/production/):

```
/minimoi/production/telegram_agent_bot_token    ← minimoi_agent_bot
/minimoi/production/telegram_system_bot_token   ← minimoi_system_bot
/minimoi/production/telegram_chat_id            ← Robert's chat ID
```

Test (`/minimoi/test/`):

```
/minimoi/test/telegram_agent_bot_token          ← minimoi_agent_bot_test
/minimoi/test/telegram_system_bot_token         ← minimoi_system_bot_test
/minimoi/test/telegram_chat_id                  ← Robert's chat ID (same)
```

Token retrieval:

```
python

def get_telegram_token(bot_name: str) -> str:
    path = get_telegram_path()
    return get_secret(f'telegram_{bot_name}_token', ssm_path=f'/minimoi/{path}/')
```

Part 5 — Graceful Environment Promotion

When promoting Mac (or Mac Mini) to production:

Step 1 — Data sync (manual for now) Before promoting, sync key data from current production to incoming:

```
bash

# From production EC2
docker exec postgres-ai-agents pg_dump -U postgres personal_agents \
    > /tmp/minimoi_full.sql
aws s3 cp /tmp/minimoi_full.sql s3://minimoi-data-332704997792/sync/

# On incoming production (Mac)
aws s3 cp s3://minimoi-data-332704997792/sync/minimoi_full.sql /tmp/
docker exec -i postgres-ai-agents psql -U postgres personal_agents \
    < /tmp/minimoi_full.sql
```

Step 2 — Role swap

```
bash
```

```
# On EC2: change to standby
# Edit /opt/minimoi/.env: MINIMOI_ROLE=standby
docker-compose restart

# On Mac: change to production
# Edit .env: MINIMOI_ROLE=production
./scripts/start_minimoi.sh
```

Step 3 — DNS update (Cloudflare) Point `app.minimoi.ai` to Mac tunnel or Mac Mini IP. Propagation: ~2 minutes on Cloudflare.

Step 4 — Verify Full checklist on new production instance before declaring switch complete. Estimated total time: 2-4 hours including testing.

Rollback: Reverse steps 2-3. Data sync direction reverses. Estimated rollback time: 1-2 hours.

Part 6 — EC2 Bot Container

Add to `docker-compose.prod.yml`:

```
yaml

telegram-bot:
  build:
    context: .
    dockerfile: telegram/Dockerfile
  env_file: /opt/minimoi/.env
  restart: unless-stopped
  depends_on:
    - postgres
  networks:
    - minimoi-network
  logging:
    driver: "json-file"
    options:
      max-size: "10m"
      max-file: "3"
```

Mac startup script (`scripts/start_minimoi.sh`) includes the bot service via `docker-compose.dev.yml`. On Mac it picks up `MINIMOI_ROLE=standby` and uses test bots automatically.

Part 7 — Documentation

Create docs/TELEGRAM_ARCHITECTURE.md:

markdown

Telegram Architecture

Two Bots

Bot
Purpose
Direction

minimoi_agent_bot
CoS + OpenClaw conversational
Bidirectional
minimoi_system_bot
Briefings, drills, alerts, commands
Mixed

Production / Standby

MINIMOI_ROLE=production → production bots, all scheduled jobs run
MINIMOI_ROLE=standby → test bots, scheduled jobs suppressed

Test Bots

minimoi_agent_bot_test, minimoi_system_bot_test
Created in BotFather. Same developer receives both.
Standby instance always uses test bots.

SSM Paths

Production: /minimoi/production/telegram_*

Test: /minimoi/test/telegram_*

Environment Promotion

See docs/PRODUCTION_CUTOVER_PLAN.md for full promotion procedure.

Data sync required before promoting. Estimated switchover: 2-4h.

Estimated rollback: 1-2h.

Commit Sequence

1. Collapse three bots → two bots, reassign all send_message() calls
2. Add is_production(), get_telegram_path(), startup logging
3. Add telegram-bot to docker-compose.prod.yml and .dev.yml
4. Set MINIMOI_ROLE in both .env files, update Mac startup script
5. Add docs/TELEGRAM_ARCHITECTURE.md

Robert Actions (before Claude Code builds)

1. Create two test bots in BotFather: `minimoi_agent_bot_test`, `minimoi_system_bot_test`
2. Add SSM parameters under `/minimoi/test/` (2 tokens + chat_id)
3. Rename/consolidate existing production SSM parameters to new two-bot naming convention under `/minimoi/production/`

Definition of Done

- ☐ Two bots only — agent_bot and system_bot
- ☐ All outbound alerts use system_bot token
- ☐ All conversational (CoS, OpenClaw) use agent_bot token
- ☐ `!ops` handled by system_bot
- ☐ Two test bots created by Robert
- ☐ SSM parameters under `/minimoi/test/` (3 parameters)

- ☐ SSM parameters updated under /minimoi/production/ (3 parameters)
- ☐ is_production() and get_telegram_path() in shared utils
- ☐ Startup log shows role + bot path on every app start
- ☐ MINIMOI_ROLE=production in EC2 .env
- ☐ MINIMOI_ROLE=standby in Mac .env
- ☐ Scheduled jobs guarded with is_production()
- ☐ telegram-bot container in docker-compose.prod.yml
- ☐ telegram-bot container in docker-compose.dev.yml
- ☐ docs/TELEGRAM_ARCHITECTURE.md committed
- ☐ Verified: system_bot sends on EC2, test bot sends on Mac
- ☐ Verified: agent_bot responds on EC2, test agent_bot on Mac
- ☐ Verified: promoting Mac to production role changes bot tokens

Commit Message

Telegram: two-bot model, production/standby roles, swappable environments.
Consolidates three bots to two, enables graceful environment promotion.

*Spec · 2026-06-20 · Claude.ai Replaces: spec_telegram_architecture_2026-06-20.md
Related: docs/PRODUCTION_CUTOVER_PLAN.md Robert actions required before build:
create 2 test bots in BotFather*